genpact

SALIENT OBJECT DETECTION USINGCNN ENCODER-DECODER NETWORKS

PRESENTED BY:
Dea Ceku

PRESENTED TO:

Data Pipeline & Preprocessing

Dataset Preparation

- DUTS Saliency Detection Dataset
- Image-mask paired samples
- 70% Train / 15% Validation / 15% Test split

Preprocessing Steps

- Images resized to 128×128
- Pixel normalization to [0,1]
- Conversion to PyTorch tensors

Data Augmentation

- Random horizontal flipping
- Brightness variation
- Improved generalization and reducedoverfitting

01	Input Image
02	Resize (128×128)
03	Normalize Pixels
04	DataAugmentation
05	Tensor Conversion
06	PyTorch DataLoader

Baseline CNN

Encoder

- 3 convolution blocks
- Conv2D + ReLU + MaxPooling
- Extracts image features

Decoder

- 3 upsampling blocks
- ConvTranspose2D + ReLU
- Final Sigmoid output mask

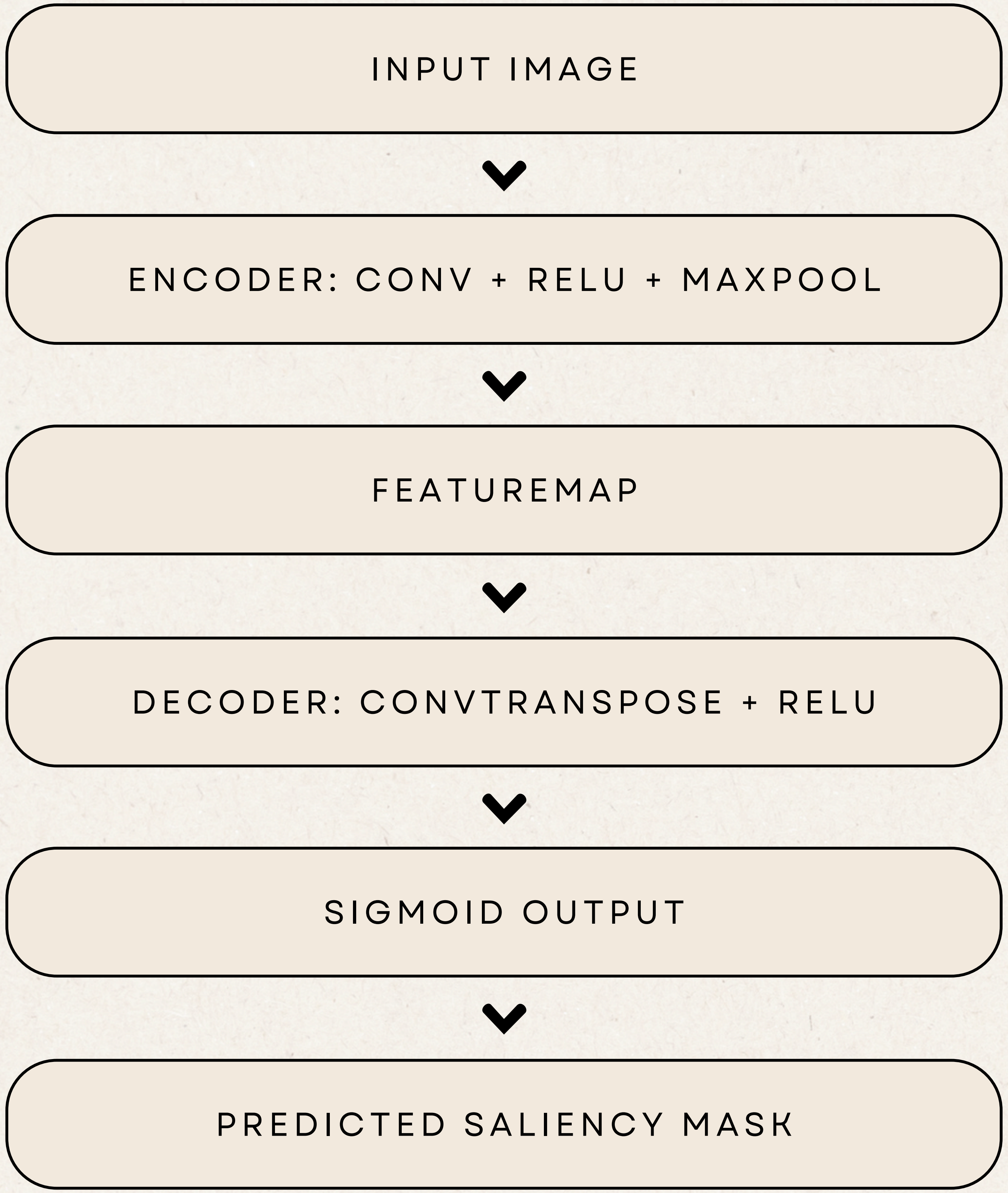
Improved CNN

Improvements Added

- Batch Normalization
- Dropout
- More feature channels: 32 → 64 → 128

Purpose

- More stable training
- Better feature learning
- Reduced overfitting



RESULTS & EVALUATION

Key Observations

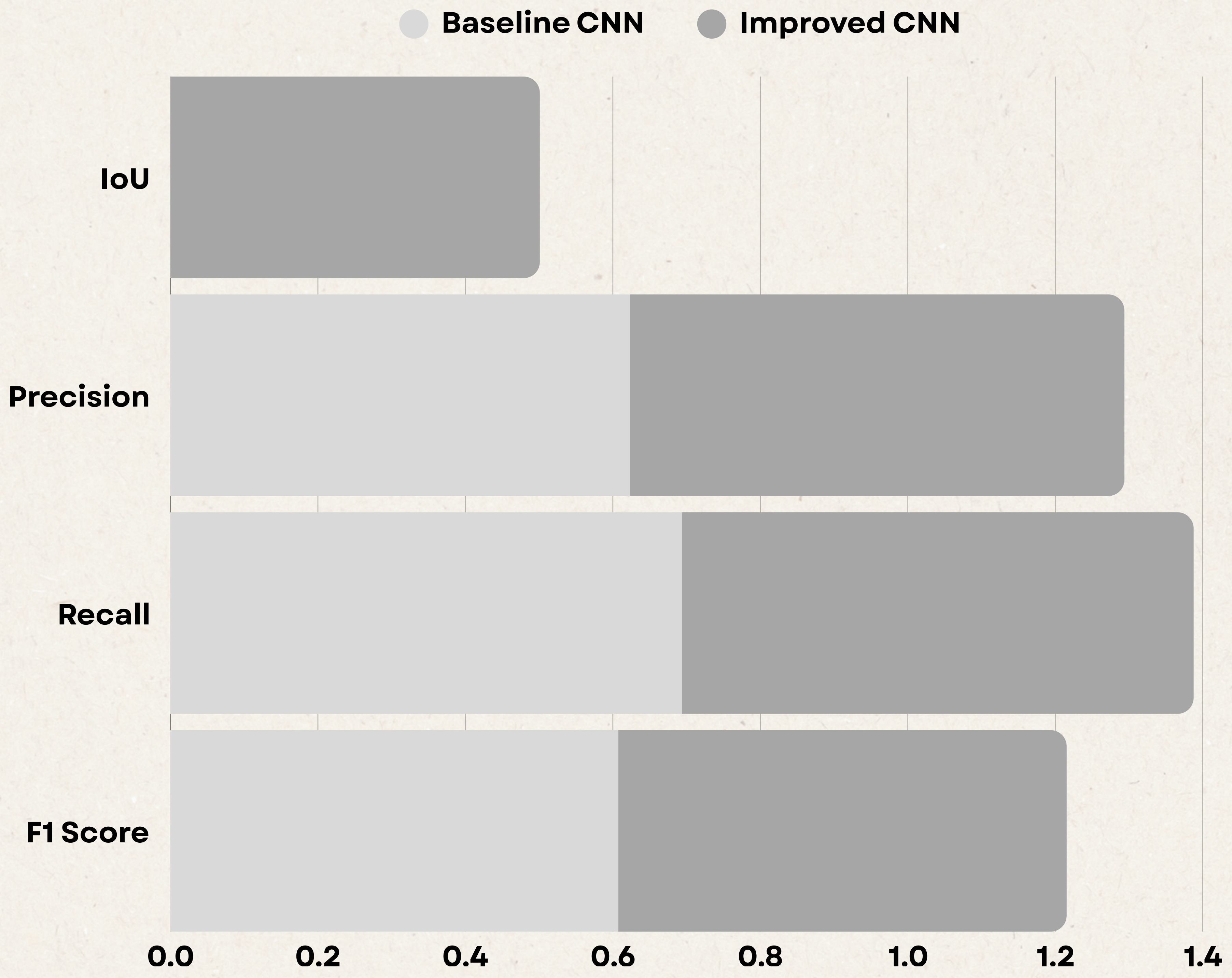
- Improved CNN achieved higher IoU and F1-score
- BatchNorm improved training stability
- Dropout reduced overfitting
- Validation loss stayed close to training loss
- Model learned salient object regions

Visual Results

- Predicted masks captured object shapes
- Overlays highlighted foreground regions
- Some blurry edges and noise remained
- Improved CNN produced cleaner masks

Demo Results

- Real-time inference on uploaded images
- Saliency masks generated successfully
- Overlay visualizations displayed predictions
- Inference time ≈ 0.03 seconds



CONCLUSION

- Successfully implemented a Salient Object Detection system using CNNs
- Built both baseline and improved encoder-decoder architectures
- Improved CNN achieved better segmentation performance
- Data augmentation and BCE + IoU loss improved training quality
- Demo successfully generated saliency masks on custom images
- Project demonstrated a complete end-to-end deep learning workflow